

CODE SPORT



In this month's column, we discuss the topic of applying NLP techniques to source code analysis.

Natural language processing and deep learning techniques have been used widely in various domains, such as improving Web search results, conversational AI systems, analysing medical records, classifying medical images, etc. An emerging area of application of deep learning and NLP is that of software development. There are a number of use cases for NLP in software development and I will provide a brief overview of a few of these applications in this month's column. For those of our readers who may be interested in reading up more on this topic, I presented a tutorial on this topic in the PAKDD conference last month, and the slides are available at <https://bit.ly/3vW1DCa>.

Before we get started on the applications of NLP to code, let us first consider whether current NLP techniques can be applied to source code. Current NLP applications are in the context of human language text. The naturalness of the human language text makes it amenable to statistical NLP techniques. The main characteristic of the natural language text is that the meaning of a word or phrase can be inferred often from its context itself – namely, the surrounding words or phrases. This fact is well known in the NLP community and often cited as Firth's law: "A word is known by the company it keeps." This forms the foundation of the statistical NLP techniques and enables us to leverage distributional semantics paradigm to build distributed representations/embeddings of words, phrases and documents, etc. This leads us to the question: Is there naturalness in software/code which will allow it to be amenable to statistical NLP techniques?

While, in general, programming languages by themselves are complex and have a multitude of different functionalities, when we analyse the actual programs that are written by humans in high level programming languages, we find these to be simple and often repetitive. Real world programs are often amenable to statistical

NLP techniques because of this inherent repetitiveness found in software written by humans. This is known as 'naturalness' of software. Given that human written code exhibits naturalness, we can use standard NLP and deep learning methods on it for building software tools. But this brings us to the other question: Since deep learning techniques typically require large amounts of data, how can we get such large data sets for code?

The past few years have seen the emergence of publicly available large open source code repositories in GitHub/SourceForge/Bitbucket. This has given rise to what is known as 'Big Code' – referring to large amounts of code that can be used for building deep learning models on it. These two factors, 'naturalness of software' along with the availability of 'Big Code', have enabled applying deep learning (DL) techniques to building models of source code in high level programming languages.

There are a number of use cases for applying NLP/DL techniques on source code. These include building smart software engineering tools for various tasks such as automatically learning coding guidelines/naming conventions, automatically finding bugs in code (also known as bug localisation), automatically fixing software bugs (also known as program repair), mining API sequence for satisfying a natural language user query, searching for code fragments that satisfy a user query, automatic comment generation, automatic commit message generation based on code diffs, code clone detection, etc.

In the context of applying DL/NLP techniques to software engineering tasks, here are three important questions we need to answer first:

1. Is there an analogous standard NLP task we can think of for the software engineering task for which we want to apply deep learning techniques? For instance, let us consider the problem of software clone detection. This can be mapped to the standard NLP task of paraphrase detection or duplicate text